Tessellated Constellation Architecture

Contracts are back, baby!

Michael E. Shaffer
github.com/Pelentan/tca-blackjack

If It Ain't Broke, Don't Fix It

An axiom to live by, but not a hill to die on.

This first section is the white-paper prerequisite of *defining the problem*. If you want to skip to the good parts, I don't blame you. I normally do. Just know this isn't me theorizing. I built a working proof-of-concept here: github.com/Pelentan/tca-blackjack

First, I'm not saying the current ways of creating applications are broken. A lot of other people would, but I prefer to say things are "not ideal". I'm also a fan of Monty Python, just to put that into context. But ever since Grace Hopper developed the first compiler, that battle has been raging on how to put together the symbols that get translated into machine code.

In the 70's, the industry had somewhat settled on what good software architecture should look like. The biggest reason no one adopted it had nothing to do with it not being correct. It had to do with the context in which it was correct. That being, it was ideal for a machine doing a set procedure. It was not correct for how humans think. It was a good concept, but one that was impossible to accomplish in anything other than a lab setting — at least not in a repeatable and maintainable fashion.

Add in the fact that not only was the number of people who could code very small, the very same mindset that made most of them great at coding also made them... Well, let's just say they were very independent thinking. As well as creative. Which led quite a few of them to develop their own methodology and swear by it. Which in turn led most businesses who hired them to go: "Yes, yes, yes. But when can you have it to me?"

This started the drift. And decade after decade, the drift increased. From time to time there were efforts to move back to a more codified architecture. But every time it ran afoul both of human nature to do what needs to be done to get the job accomplished and the axiom I stated above. What starts as "patch for now, rewrite later" becomes another patch. And another. And another. Until the codebase becomes too monolithic to even consider trying to reign it in, much less rewrite it correctly.

Add to that the plethora of new programming languages that kept coming out. Some actually new, others variations on existing ones that couldn't play nice even with the parent. Almost all of them were created to solve a specific problem that programmers were coding to — one that the current favorite couldn't do well. Then, once it was working, it kept getting added onto in order to share its goodness with everyone. Bolting libraries that tricked the core code into doing things it had never been designed to do. With most developers interpreting what good architecture was in their own way.

Let's look at one architectural tenet that almost certainly was around in people's minds for a while, but the first time I could find it written about was in *Design by Contract* by Bertrand Meyer in 1986. That is, of course, the Contract. His version was different than the current OpenAPI version really only in where the enforcement happened. It was designed to enforce how a routine is communicated with — exactly what type of variables are required and accepted, and exactly what type of variables are returned.

This is beautiful. In concept. Everyone knows what to expect from everyone else. In theory, it would make it impossible for miscommunication to occur if you write to the Contract. Only... the human brain was never wired to be able to strictly adhere to any contract. We are, by nature, creative and curious. We need to try new and different things. Find unique ways of doing things. And writing a contract before I know what I'm going to code? Are you daft? With the latest

version, OpenAPI, you can get developers to write the Contract well after the code is up and tested. If you're lucky. And updates? They may or may not follow that Contract. It's not right, but it is reality.

Then there is security. I won't spend much time on that. Just a nod to the fact that it's a constant race to keep up with. More applications means more lines of code means more chances of a small mistake to leave everything wide open. Sammy Azdoufal and the DJI Romo vacuum, anyone?

So, that's the landscape as it is now. Working, but far from perfect. One could argue it's impossible to even keep up at this point. Tech debt continues to pile up, patch after patch. It's getting more and more expensive to even *plan* a total rewrite, which needs to happen for at least 90% of the code out there. We *are* headed for a crash. Time to take control.

Enter AI.

Everyone has their own idea of what AI is, what it should be used for, and what its dangers are. I'm going to suggest you look at it slightly differently.

AI is A-0 4.0. And if you don't get the reference, allow me to show you.

What is Tessellated Constellation Architecture (TCA)

> *Contract is king.*

First, let's start with the name. Like all good names, it encapsulates the concept.

Tessellated. This describes a pattern made with either tile or stones. These stones are cut into a small number of fixed, predetermined shapes. Those shapes are used to form the desired image. That shape determines how they will fit as the goal is to have no gaps. That shape is their "contract". The stone is something that does a singular function. In the case of tile, its job is extremely simple. In this architecture, the Job is a singular service. The boundary that contains it is... well, a container.

Constellation. This is a set of stars and galaxies that when you connect them they form an image. Key to this name over others is that those stars aren't actually clumped together. Normally each point is hundreds, thousands, millions of light years away from the others and separated in all three dimensions. The Job containers of TCA obviously can't have that level of distance between them, but the Job that does the calculations could be on the other side of the planet from the Job that writes the results to a database. Modern distributed networks make the connections only slightly less ethereal than the lines drawn by our minds. And the image is, of course, a fully functioning application.

Architecture. First codified by the Roman Vitruvius, around 15 BC. He wrote the De Architectura, a ten-book series codifying... Oh, wait. You didn't want me to go that far back. And you all know what architecture is in this context. So, I'll skip it. Instead, let's dig into the actual structure. Note that a total of zero of these are new inventions. They all have a fairly long history. The "new" I'm bringing is only how to bring them all together. And in the darkness... No wait. Properly and ethically utilize them to their fullest potential.

The Contract

> *Your finger, please.*

I mentioned Contracts in the "problem" section. We've tried the concept over and over again, and it keeps slipping. The "how" TCA keeps it from slipping this time is AI. In TCA, the Contract is something that can never be written after the code. This isn't documentation. This isn't a README. It's a machine-readable, verifiable, and enforceable YAML file. It's what controls what crosses the boundary in either direction. Most importantly, it is immutable once initial development is done. If any change needs to happen, that's a brand new Job. If you're wearing suspenders, think C++ functions.

TCA uses OpenAPI as the Contract structure. It delineates every endpoint, every request and response shape, and every status code. You've likely seen these in action if you've seen Swagger. What it does not, and cannot, specify is what is inside the Job. Not even the language.

This is where AI first gets active in the build process. Have any of you ever built an OpenAPI Contract? Give it a try and you'll understand why they are pretty much written after the code. If then. With TCA, you will have "discussed" what you want the application to do and how with the AI. At this point you discuss what Jobs need to be created. You get extremely specific as to what information you want to feed the various Jobs and what you need out. You'll have some back and forth, tuning it up. And then you have it write the first batch of Contracts. There's a reason these initial ones all need to be written at the same time — the AI will have to do some tweaks in-flight to get them to all work together. This gives you the initial framework that everything builds off of.

Feel free to look at the Contracts in the repository at **contracts/openapi**. Every one of those was written by Claude *before* the first line of code for a Job was written. That was the bible. Only minor changes were made as some Jobs were created and needed modification. But this, the initial development stage, is the only time that's allowed. Ask the guy wearing suspenders next to you why.

> *Reference: Design by Contract (Bertrand Meyer, 1986) — preconditions, postconditions, invariants. The formal philosophical foundation. OpenAPI Specification (2016) — the practical modern expression in distributed systems.*

The Job

Git 'er done!

The Job is the smallest unit that TCA cares about. TCA has zero knowledge of what's inside of it and wouldn't know what to do with it if it did. In reality, it's a set of code that does a specific thing inside a container. TCA just knows that it can feed in a set of variables and, like magic, it gets a set of answers back. And by the same token, the code inside the Job layer knows absolutely nothing about the outside world. It just knows that at times some disembodied voice delivers some words from heaven. It then knows to take those words and convert them into actions. Then to finally shout the results back up to heaven. At that point, they go back to the checkers game they were playing.

And what I mean by "does a specific thing" is that they only do a single, fixed, set of actions based on the words given. Think of it like an assembly line. It cannot rely on anything outside of the job with the sole exception of a database hooked to it. Other than that, nothing.

There is one last, key thing about Jobs. They should be at most a few thousand lines of code. If it's longer, it's probably doing multiple jobs. For TCA to work best, smaller is better. A developer should be able to give it a quick glance and understand everything they need to know about the *current* code that is doing the job. More importantly, it must fit into an AI's context window.

Some people would say this is just a microservice. And they wouldn't be entirely wrong. Pretty much same concept. It's just that up until now, a full set of them has been almost impossible to maintain, or would have degraded until they are just function calls from other parts of the application they are embedded in. TCA changes that. Though I'll still take my two pizzas, thank you.

This is the next area that AI gets its hands dirty. The AI has the Contract in its "mind". It knows it can't violate that. It also has expert knowledge of pretty much any language you want to program in, as well as all the approved best practices in coding and security. You have it write the Job. You still need to be competent enough to take a quick look at the code to make sure it's doing what you need. Including doing the compile to make sure nothing jumps out at you like deprecation warnings. But that just means you go back to your AI and have it make the corrections. For the duration of the creation, the AI is the source of truth for the code.

> *Reference: Unix Philosophy (Doug McIlroy, 1978) — do one thing well, write programs that work together. Information Hiding (David Parnas, 1972) — the module knows nothing the consumer doesn't need it to know.*

Polyglot Implementation

Grab your babelfish.

Multiple languages inside an application have always been a necessary evil of most projects. You not only have the issue of needing experts in each language and database system used, you also needed an expert in getting them all to fit together. However — and some of you likely already put this together — Job containers bound by written-in-stone contracts turns this evil into a most desired feature.

Each Job is in its own container. Everything in and out is constrained at that layer by the Contract. Suddenly, the choice of what language you write the job in — or what type of database you're creating the interface Job for — goes from "what most of us know" to "what is the best choice for this Job?" Partnered with AI, no one on the team has to have any experience with a given language. Just solid skills in any modern programming language.

> *Reference: Polyglot Persistence (Martin Fowler, 2011) — different storage problems need different storage technologies. TCA extends this principle from data storage to implementation language.*

The Three-Year Lifecycle

Welcome to Carousel!

If you don't get that reference, don't worry. They'll be remaking it soon. However, in a nutshell, I'm proposing that almost no Job survives more than three years without a complete rewrite. Before I dive into the "why's", let me restart some hearts.

In the proof-of-concept I just completed, I proved out the feasibility of it. I had a Bank Job (no, not that kind). Originally, we decided to write it in Java due to floats being a compile error in that language. We got about halfway through and decided to add some functions to the original Contract. And kept running into library issues. Nothing major, just annoying. So, I sat down with Claude and asked if we should shift languages. I made a suggestion. Thirty minutes later we had a COBOL-Go Job, compiled into pure binary, up and running — replacing the old Job without changing a single thing in any of the other Jobs.

That was thirty minutes from "this isn't working like I want" to a completely new codebase in a whole new language built by Contract into a Job. Slotted in and running. If you've caught your breath, let's jump into the why. Though I suspect most of you can rattle them off faster than I can.

Change. Everything changes rapidly. And most of those changes are breaking changes. Code, libraries, networks, physical machines, abilities. And a new one that is even more scary: ideologies. Looking at you, Rust. I don't care where your loyalties lie. Should you have to worry about where your vendor's lie? The landscape is constantly changing. Code needs to as well.

Vendor lock. If you start with one language or set of libraries, due to the nature of the IT landscape now, you are pretty much locked to it for the life of the application — which, if it's written well, is a decade or more. I could probably write a couple of papers on why this is "problematic" at best. But at this point it's giving way too much power to salespeople who make their pitch to someone up the chain who really doesn't have much coding experience.

The age of the code. I'm willing to bet that for most of you, your newest code is a patch for a hotfix for a patch on code that is over a decade old. When's the last time you got a real new feature out to your application that wasn't just another flavor of what's already in there?

Written-in-stone Contracts + Job containers + AI makes rewrites an afternoon of work, from concept to ship. A full feature development cycle takes a bit longer. Say five days.

> *Reference: Logan's Run (William F. Nolan and George Clayton Johnson, 1967) — units are assigned specific jobs. They perform these jobs for a set number of years and then are renewed in carousel.*

What Was Missing

I've touched on it throughout. The answer is AI. All of the above ideas and principles have been around for decades if not longer. Each one does what it needs to do with precision. But they were all written for machines. They codified things in ways machines could understand and utilize. But those methods were still outside the way the human brain functions. In some ways it's like fusion. We understand fusion. We can even make it happen in very fixed laboratory environments. But we can't do it at scale and we can't do it profitably. AI changes that.

However, I put forth that the way I've seen everyone — and I do mean everyone — try to use AI for coding has been wrong. Not just incorrectly, as demonstrated by all the failures and "vibe coding" examples. But wrong as well. They are trying to use a hammer to drive a screw. Everyone is focused on trying to get AI to replace a skilled coder. That is the wrong dynamic. Not only won't it ever happen for very human reasons, it physically can't happen. As someone who built their first computer with a soldering iron and a wire-wrap tool, you might want to take my word on that. Current technology isn't even close.

So what is the right way to view it? It's what AI actually is in its current state. Like I mentioned in the problem section — it is A-0 4.0:

A-0 (1952): Human writes symbolic math → compiler → machine executes

A-0 2.0 (FORTRAN, COBOL, C): Human writes closer to English → compiler → machine executes

A-0 3.0 (Model Driven Architecture): Human writes formal specification → generator → machine executes. The aspiration was right. The tooling was never adequate.

> **A-0 4.0 (AI):** Human writes intent in natural language → AI → working code → machine executes

With apologies to Grace Hopper and her work, I'm going to shorten it to A-4. Sometimes looking good needs to trump technically correct.

The proper use of AI in IT is as a fourth-gen compiler. Not something that replaces a skilled engineer, but one that enables them to achieve productivity, fluency, and precision that they just can't achieve with their hands alone. And it's not just tool use. With how AIs are constructed, you're only hurting yourself when you don't treat them like a partner. I wrote another paper on that, but in a nutshell you'll get much more out of them if you treat them like a junior developer who can not only write almost perfect code in almost any language, but can also help you with research. Additionally, it's a lot less wearing on our human sides.

The key is that both the human engineer and AI coder-writer have strengths and weaknesses in different places where the strengths of one make up for the weaknesses of the other. An AI never gets tired. Never forgets (up to a point). And can compile multiple documents down into working code. A human can envision what is needed. Can plot out how it's likely to be used. Has real-world experience on how things are put together. And won't hesitate to push the AI out of the way to work on a piece of code they think they can innovate on. The AI is just working on what has been created before. The human works on what hasn't been created yet.

> *Reference: Model Driven Architecture (OMG, 2001) — the aspiration that tooling would generate faithful implementations from formal specifications. The aspiration was correct. The tooling was never adequate. AI is the first tool that makes it genuinely viable at production scale.*

Proof, Not Theory

> *Show me the money.*

All the above may sound great. Or it may sound horrible. It's just theory, right?

Not quite. github.com/Pelentan/tca-blackjack — please feel free to open it up and follow along.

I chose blackjack for a couple of reasons. One was because if you're a coder, it's likely one of the first programs you wrote. It gives you a taste of all sorts of factors of coding, but is a very simple counting game with established rules. However, the big reason I chose it was because it was comprised of several very delineated jobs that anyone who's played blackjack would understand. The goal was a fully functioning blackjack game where all the different jobs were separated out into separate Job containers, and each one was written in the language that best supported it. It was meant to be an extension of another project I did to prove out my concept of Engineer-AI partnership. It evolved into TCA.

What Claude.ai and I created:

- 12 OpenAPI 3.1 Contracts
- 12 containerized Job services
- 6 languages (7 if you count the Java Job we ditched half-way through)
- 3 Postgres databases

- 1 Redis database used for a message queue
- Approximately 10,000 lines of implementation code
- Full mutual TLS across every internal service boundary
- WebAuthn passkey authentication with mandatory enrollment
- Real email delivery via SMTP
- Real-time observability of all inter-Job communication

One senior engineer + Claude.ai · Under 40 hours

Ignoring that it's "just" blackjack, we estimate that a traditional team would budget three to four months for a system of this complexity. Claude and I did it in under forty hours. Let me show you how we did it.

Defining the Project

This is where treating the AI as a partner comes in. Remember, this isn't just a computer answering you. It's also the thousands of other programmers whose work was compiled into the AI. I know a lot of programmers like to write or type out their ideas so they can mull them visually. With an AI, it's that on steroids.

Explain to the AI what you're trying to accomplish. Discuss what you think the different Jobs should be. Let the AI know what languages should be used. Let it know what the criteria for both the Jobs and the application are. Then ask what it thinks. And iterate. Explore. Brainstorm different things. If you've chosen a good AI, it will push back on things that don't work as you think they should.

In the end, you should have come up with an initial set of Jobs and what languages those will be written in. Time for the next step. However, this step should never be considered a one-and-done. If at any time something doesn't feel right, come back to it and hash it out again.

One of the principles I established right from the beginning was that this was going to be as close to zero-trust as possible and still run as a demo. While a lot of the security was going to be put in place after a working application was in place, the structure for zero-trust needed to be built in from the start. This meant stubbing out the security Jobs so they returned an "accept" every time they were queried — but only after ensuring that they were getting the right types of data.

Additionally, while this was going to be a working application, for development purposes I wanted a test loop that made sure everything was working as we built it. Which meant a demo-player.

Commit Zero

A bit of honesty here. For this project, this was *not* commit zero. As you can see if you look at **infra→certs**. That is why this is now carved-in-stone commit zero. Some of you probably already do this. I sorta always do it. I, as a human, got sloppy because I hadn't made that mistake in years. Now it's part of TCA.

- Before a single line of code is written, a .gitignore is configured for the full, expected, polyglot stack.

- CodeQL security scanning active for every language in the project via Actions.
- GitHub secret scanning enabled.

These are not suggestions. These are check-boxes. Security scanning at commit zero means every subsequent commit is scanned. Adding it later means every prior commit wasn't.

Contracts

Another bit of honesty here. When I started, I wasn't thinking about Contracts. I knew about OpenAPI. But in all the shops I had been in, it was always something worked up after the application, or added feature, was complete. In the Defining the Project session with Claude, I had said I wanted to fix the ins and outs beforehand and lock them in. My thoughts were more like the C++ functions I mentioned earlier. It was Claude that suggested we start with OpenAPI contracts. I was like: We're allowed to do that? Guess you can teach an old dog new tricks.

So that was the next and first step before each Job was written. We worked out what data the Job was going to need and what data the Job was going to provide. And then Claude wrote the Contract. I highly recommend you take a look at a few of them at **contracts/openapi**. Including **hand-evaluator.yaml**.

For one, it may look like it's also defining some of the internals of the job. But it's just actually telling whomever writes the code — human or AI — what it's going to get.

Then take a look at the "servers" block. You may be going "Uh-oh. Hard-coded URL!" Yes, but no. What it's referencing is the *service name*. That will always be the same, no matter how many containers of this Job are up or where on Earth it is. If this Job is unreachable, then it's a failure of architecture, not code.

Contract carved in stone.

Creating the Jobs

A quick reminder. For the duration of development, the AI is the source of truth for the code. The human can tweak things and adjust things to see if they work, but these then need to be fed back to the AI so it can add them to the code. You can very likely come up with better ways of doing various things. The AI will know every place that will break things. You will likely run out of room on one or more conversations. If you're using Claude Projects, Claude has decent understanding of other conversations in the Project, but for each new one, you'll want to upload all the code in a zip file. This makes sure the new conversation has all the latest changes.

What I did at this point was confirm with Claude that he understood what I was wanting out of the Job and that he had the Contracts "in mind". Then asked him to code it out for me — working Jobs, stub Jobs, demo reel, and all. It took him a while. I went and got coffee. When I came back, he had the first set of code all tarballed up for me.

Lesson learned: If you have him do more than one Job, run the compile for each one individually. I was feeling froggy. I just ran docker compose up — just to see. I think we were at 10 jobs at that time. Most compiled without issues. Others had issues. Others crashed. So I had to go back through and recompile each one individually to pull out what wasn't right.

Then began the back and forth. One thing I had told him was that before each Job was created, I wanted him to go out and make sure he was using the latest and greatest. He initially took that

as a soft guideline. I had to reiterate that a few times. Got it locked into the **work-methods.md** file now. But this was always caught when I watched the compile and build of the container. Maybe not the first time, but I'm pretty sure we got them all. I'll touch on this further when I go into why I used Claude "chat", not Claude Code.

If you look through work-methods.md, it specifies what the AI is to deliver back on these back-and-forth cycles. It's supposed to be a tarball with all files and the command to extract it in the root directory, as well as the build command for the updated services. It took some work to get Claude to settle down to this — he always wanted to be more helpful than needed. But the last couple of days, he stuck to it. Except when he couldn't. There were times when it had to be changed, which immediately drew my attention to what needed changing. A subtle: "Hey, meat-sack. Pay attention to this." But he knew that I always used docker compose up so I could watch the feed. Which meant I would be bringing it down before the tar file extraction, waiting for the build to finish so I could catch errors, then bringing it back up myself.

Normally, if it didn't compile and run the first time, it only took a couple of back-and-forth trips to get it there. A couple of times we did run into a case where a variable type coming in wasn't the best for what the language could handle. Instead of "working around" that issue, we went back and updated the Contract and any Jobs that were currently sending to that Job.

There was one exception that needs to be looked at in greater depth. Not because anything was "wrong", just that it wound up proving out the Contract→Job concept. Taking it from theory to proof-is-in-the-pudding.

The Bank Job (not that kind) was originally written in Java. There were a number of reasons, all valid. But we kept on having issues with it when we went to add functions. It was known from the start that this Job was likely going to be morphing a lot. The functions we had for the Job sometimes wanted to fight each other over which library to use. Each issue was resolved with one or two updates. But it was happening on almost every change.

So I took a step back and went back to the Defining the Project stage. I suggested two options for replacing Java. The first was C or C++. Claude politely shot that idea down. Gathered the remains and set them on fire. And put the fire out with gasoline.

Okay...

The other suggestion he liked: COBOL with an interface language. We both knew that the primary issue COBOL had was interaction with basically anything not COBOL. But COBOL is still the reigning champ at bank-type operations. He suggested writing a Go interface which called COBOL functions — and everything can compile down, Go and COBOL, into a single image.

We started hashing things out. The Contract would need zero changes. He warned me he didn't have much internal knowledge about COBOL; it was a "dying" language after all. I said that was fine, this was exploration, and his knowledge was more current than mine by a few decades. Give it a shot.

Time spent from the point of my taking us back to the Defining the Project stage to the swapping out of Jobs with the new COBOL-Go image that has worked perfectly ever since?

30 minutes. And that's with a break to get coffee.

Zero-Trust Baked In

I think this one little factoid demonstrates the mindset: For internal communications there are twelve, *different*, mTLS certificates securing *all* internal service-to-service calls. I can hear a lot of the wails from the non-Security folks. "That's too complex! It takes too much time to implement!" Claude and I built the prototype up to the point where we were going to build out the single-player game with all comms in the clear. Everything was Contracted and built for it. Just basically stubbed in. I had Claude go back through and implement it across 12 Jobs, 6 different languages. Took about 15 minutes, counting extracting the files and rebuilding everything.

Let that sink in for a minute. The *internal network* has mutual TLS connections between each service. No more depending on the network layer for defense. No relying on third-party server meshes. Hard security baked in, utilizing certs that can be rotated automatically and issued by your organization's existing certificate authority.

A side note: this was one of the things that required an update to the Contracts. Though to be fair, it wasn't a change — it was the addition of the block that controls transport-level security. Which... OpenAPI doesn't have specifications for. Yet. But any AI should get to that section and go: "Hmm. Humies are trying to be creative. I think they want me to do something here. Ok. I get it." You may also note that some of the contracts have specification for TLS 1.2 minimum instead of 1.3. This was because the libraries for some of them for local-loopback, used for health-checks, only support 1.2. This is something to improve on in the next go-through.

Requiring Passkey authentication is almost an anti-climax. It's used more and more. However, there is still resistance to it based on how difficult it can be to get set up on the application end. A valid argument. That was the one task that took us the longest to get working. And that was with Claude having all the documentation, libraries, and specifications. Multiple cycles of back and forth for the different Jobs that needed to be interacted with. I think that one took a good hour. Though now we have working reference implementations to work from for future work. In addition, the framework in place doesn't care whether the verification comes from the device or a physical card. That change is a minor one.

The payoff is not just the security, but the ease for the user. The path goeth thusly. New user registers by submitting their email address. They get an email that has a verification link with an embedded JWT token. They click on that link and that JWT is retired from the universe. They are taken through the process of setting up a Passkey — which consists of clicking a button then responding to whatever their device is using to verify their identity. In Windows, that's the PIN they log into the computer with. They are issued a new JWT that only works for playing the game. That's it. What's not to love?

The final level of security is one that I feel isn't getting the attention it deserves. And that's an AI writing the code. If the Guidelines file is set up correctly, an AI will be able to write consistently more secure code than any human can. Maybe not on a one-page app. But AIs are machines. They don't get tired. Their eyes don't glaze over. And they never say "This will work for now; I'll come back later and fix." This doesn't mean the Engineer can hand off responsibility. They still need to scan the code. Watch the build. Make sure nothing deprecated sneaks in. But that just goes back to security in depth.

One thing of note that may seem to violate the zero-trust is that the connections to the actual databases in the application rely only on network isolation in this proof-of-concept. That was a conscious decision — in an actual app going to production, most if not all of those would be external to the TCA application that was built. The tying in of those will have already been set and would require updating the code.

"Yes, Security Team. Santa Claus does exist."

The Observability Layer

This one is an interesting one. It started with a whim and became a required feature. I had told Claude at the beginning that I would like to see the calls between the Jobs. We hashed out what I wanted to see, keeping things secure. I was thinking just a fun little thing to watch while we developed. Something to be taken out later. Claude took me seriously. The security geeks better be salivating at this.

Normally, for communication inside an application, you have to rely on coders to put log statements at various places. Not now. While this Job doesn't provide the level of visibility you may fantasize of having, you now have observability on all traffic between different Jobs. All secure with mTLS, but visible with the right credentials. What Claude and I built was authenticated, encrypted, observable inter-service traffic.

Each Job reports all outbound calls to the Observability Job. Look for the **x-tca-observability** block in any of the contracts — well, except Observability itself. Observability then sanitizes the incoming data and feeds it to a Redis pub/sub channel. In this project, just the Gateway subscribes to it for a cool window showing the events: a drawer in the UI showing real-time every inter-service call as it happens. But there is also an **/event** endpoint already created. So yes, Security — with a service account you can get the feed from that. Sanitized and formatted before it even gets to Redis.

And like everything else, this is defined in the Contract — in another new section for the OpenAPI standards: **x-tca-observability**. A Job must be built with that already baked in.

Claude.ai Instead of Claude Code

Now seems like a good time to go over that choice. I've been exploring things with Claude.ai for over a year now. I've done a little bit with Claude Code, but I have not been impressed. In a nutshell, Claude Code feels like just an Agent for Claude.ai. It's more nuanced than that, so let's go through my full stack of reasons.

One of the primary reasons goes back to the paper I wrote on utilizing AI and to things I've referenced briefly in this paper. This new era we're moving into is going to mean AI is part of our workflow. AI is a tool, yes. But it's human nature to anthropomorphize pretty much anything we work with for any length of time. You can either try to reject that natural tendency, or carefully lean into it. As long as you keep in mind it's just a machine, no harm. And a major benefit to doing this was born out by the two 40-hour projects I completed with AI. At the end I was still pretty exhausted — it was work. But I didn't feel it was a grind. By using the "chat" version, I could take a quick break with Claude and talk about something unrelated. Then get back to things. Not something you can do with Claude Code.

Then there is the fact that CC is automating a lot of the things you, as a conscientious coder, will want to be watching. Above I mentioned the compile and build. But it goes beyond that. With most chat AIs, you can turn on a "show thoughts" mode — meaning you can watch the process the AI is going through to get to its solution. I can't count the number of times I hit "Stop" because I saw Claude going off in the wrong direction. Or worse, doing exactly what I said, and not what I meant.

Which brings us to the most important weakness in Claude Code. It *can't* reason. It just uses what it knows and builds and commits. You can't do the back and forth I mention in multiple

steps. CC is just building off old data. TCA is built around *collaboration* with AI. Not just sticking a piece of sheet-metal in a press and hitting "Go".

Claude's "chat" version provided everything needed to make that happen. And provided natural checkpoints at every step to help ensure what was built was what was needed.

Looking at the Numbers

Let's take a look at what was built.

- **12 containerized Jobs/services** using 6 different languages, including COBOL.
- **3 Postgres databases** tied to specific jobs.
- **1 Redis deployment** for pub/sub feeds and short-term data storage.
- Isolated internal networks for Jobs.
- **Full mutual TLS** across every internal service boundary.
- Approximately **10,000 lines of code**, written to the latest specs, including COBOL.
- **12 OpenAPI 3.1++ contracts.**
- **WebAuthn passkey authentication** with mandatory enrollment.
- **Real email delivery** via SMTP.
- **Real-time observability** of all inter-service traffic.
- Ability to completely rewrite the code for any single Job in **a single afternoon.**

One senior engineer working in collaboration with Claude.ai. **40 hours.**

I asked Claude what he thought it would take a traditional team. Three to four months for a system of this complexity and architectural coherence. Maybe. You tell me. Keep in mind, the fact that it was "just" blackjack is irrelevant. The meetings alone hashing out responsibilities would have taken a good chunk of that time. And a major chunk of our 40 hours was building out the TCA process itself — testing the principles.

But let's make sure there is no misunderstanding. The only developers I'm keen on displacing are those who treat IT as a paycheck. Not that I lay any blame on them. They filled a necessary role up until now. But they don't have the drive necessary to make an Engineer-AI partnership not just work, but explode.

A team of skilled engineers, paired with AI and implementing Tessellated Constellation Architecture, is The Way. Maybe that will change in five years, but we live in today. Once you have that pairing up and running, you go from a team of engineers building and maintaining a codebase, to a team of engineering **teams**. Each individual Engineer-AI team able to do 80% of the work of that original team of engineers. At that point, good luck keeping them busy with the new features you want.

And in some respects, I've been saving the best for last. This isn't just for new applications. It is the way forward for any legacy application. This includes the banking ones that they've been trying to modernize for decades. The key is this:

The language is not the issue. The architecture is.

TCA is the answer.

1. Identify specific Jobs in the application.
2. Create specifications. Decide on best language — even if it needs an interface on top.
3. Have the AI create the Contract.
4. Have the AI build the Job.
5. Point the application to it and refine as needed.
6. Rinse and Repeat.

Let's Put This All Together

Connie Wr4nglers, mount up!

Reading through all of the above, you might get the impression that I'm proposing radical new ways of doing things. Even though I've dropped pointers to the past. Some may even be impressed with what I've built with TCA. But honestly, I feel more akin to Newton than Einstein. At most, I lean on the shoulders of the giants who came before me.

When I started on this project, the name I was using was "swarm" — as in a swarm of containers that just did single jobs which combined produced a full-functioning application. The focus was to prove out Engineer-AI partnership. I knew that it worked great for small projects. But how can it tackle massive applications, including those that have been gathering patches to the codebase for decades? The only real answer was to break the application up. That led me onto each step. The development of each pillar of this architecture. Until it was up and running, proven to be repeatable and maintainable. Though only with AI.

I stepped back and looked at what I had created. And I'm embarrassed to admit that for a moment there, on the evening of day 5, I thought I *had* invented something. But then I started looking closer, and had Claude confirm. Almost every part I built into TCA had been either invented or envisioned before me. I was just running off half-remembered things I had read. Oh well. I do need to be able to fit my head through doorways.

But what I *will* lay claim to, without any shame, is pulling all those ideas together, parsing out the good and needed parts, and combining them all together in a cohesive whole. Something that the IT community has wanted to do for a long time. The big difference now being functional AI. And the idea that the AI isn't there to write the code — though it does that. But by using it as A-4: The next generation of compilers.

Mount up, CW. Time to wr4ngle your first constellation. The future isn't coming — it's already ro11’n.

A Personal Note

Yawn. Skip!

I've given some nods to how long I've been involved with computers. I'll give one more: the first program I entered was with 8 switches and a Commit button. But that isn't to set me up as an expert on all things computer or programming. Software engineering is not even something I built a career on until 2008. Before then it was just projects I picked up on the side and kept up with. My life took me all across the spectrum of available jobs. Starting with growing up a farmer, first job a dishwasher, Army as a Cav Scout (yes, I did manage to work computer work into that job), and telecommunications. Even one year as a bookkeeper. I became very good in

each job I took on — enough to cause some resentment from those who had been doing those particular jobs a lot longer and I would come in and seem to pick it up in weeks or months.

What was missed was I wasn't better than them. I was just leaning into that which was natural to me. A desire to pull things apart to see what made them work. A need to find a way to put them back together in a way that made them "better" or do something different. Learning to code was less likely to burn down the house than rewiring radios or old TVs.

BTW, yes, you can make a rudimentary arc-welder from two number 2 pencils, an extension cord, and house power. Unfortunately, my exploration into what you could do with that was cut short by "upper management".

How does all this relate to TCA? That same moving from radically different job to job meant I always had to shift my perspective. To the point where I embrace the title of JoAT. Only not "Jack" — but rather Journeyman of All Trades.

Which means when something new, like AI, comes onto the scene, I give it a good look. In this case, since I grokked at a level almost no one today can how it had to work, I was more skeptical than a lot of people. Though not for the reasons of most skeptics. Having put together computers with soldering iron and wire-wrap tool, I knew what the physical limitations were.

But that didn't stop me from exploring it and taking it apart like I do with everything. And that's where the biggest benefit of being a JoAT came in. Not to invent something completely new. But taking things and ideas from all those different areas to recombine things already in place in a new way. And spot that AI, used correctly, was what could bring them all together. Take them from "gee, it would be nice if we could all do this." To something that not only works, but I honestly feel is the way things are going to have to be done. Not in the future. Now.

Speaking of now, let's visit some of those who laid the groundwork. Who did all the heavy lifting and creating so I could come along and say "maybe if you move this here". There are many more. These are just a few that contributed directly to the concept of Tessellated Constellation Architecture.

The Giants

Almost every principle in TCA existed before TCA had a name. The synthesis is the contribution. The giants provided the building blocks. They deserve explicit credit.

Grace Hopper *A-0 Compiler · 1952*	***The machine can do more than arithmetic. Give it a program.*** The intellectual ancestor of everything in this paper. Hopper proved in 1952 that a machine could translate human-readable symbolic language into executable code — against the consensus of her peers, who were certain computers only did arithmetic. Every compiler generation that followed is her thesis extended. TCA's proof-of-concept runs COBOL. Hopper was the primary force behind COBOL. The lineage is unbroken.
Doug McIlroy *Unix Philosophy · 1978*	***Do one thing well. Write programs that work together.*** The cognitive encapsulation principle has its clearest ancestor here. McIlroy articulated in 1978 what TCA Jobs embody. The philosophy was right. The

	tooling to compose Jobs faithfully across language boundaries took another 45 years to arrive.
David Parnas *Information Hiding · 1972*	***The module knows nothing the consumer doesn't need it to know.*** Every TCA Job is a Parnas module. The developer implementing it knows its contract and nothing else. The consumer knows the contract and nothing about the implementation. Parnas's paper was foundational and largely ignored in practice. TCA makes it structurally unavoidable.
Edsger Dijkstra *Separation of Concerns · 1974*	***Different aspects of a problem belong in different places.*** The principle that justifies why a Job never needs to know another Job exists. No component should carry knowledge that belongs to another component. Dijkstra said it. The industry agreed. The industry then built monoliths with shared databases and called them microservices.
Bertrand Meyer *Design by Contract · 1986*	***A formal obligation between caller and implementer.*** The philosophical foundation of everything TCA does at the contract layer. Preconditions, postconditions, invariants. Meyer was right in 1986. The idea never failed — the implementer did. Human engineers drift from contracts not because they don't understand them but because implementation cognition works differently from specification cognition. AI doesn't have that problem.
Melvin Conway *Conway's Law · 1967*	***Systems reflect the communication structure of the organizations that build them.*** TCA inverts this deliberately. Jobs are designed so developers don't need to communicate across boundaries — which means organizational structure cannot leak into system structure. Conway described a constraint. TCA treats it as a design target to be inverted.
Martin Fowler *Polyglot Persistence · 2011*	***Different storage problems need different storage technologies.*** TCA extends this from data storage to implementation language. The principle is identical: the tool should match the problem, not the team's familiarity. Fowler made the argument for storage. TCA makes it for the entire implementation stack. AI removes the organizational friction that made true polyglot development impractical for human teams.
Object Management Group *Model Driven Architecture · 2001*	***Generate faithful implementations from formal specifications.*** The aspiration was correct. Systems should be specified formally and implementations generated from those specifications. The tooling was never adequate — code generation produced brittle scaffolding, not production systems. AI is the first tool that makes faithful specification-to-implementation genuinely viable at scale, in any language, without scaffolding.
OpenAPI Initiative *OpenAPI Specification · 2016*	***Machine-verifiable interface definitions for distributed systems.*** The practical format TCA contracts live in. Successor to Swagger (2011) and the spiritual heir to WSDL's contract discipline without the WS-* stack's complexity. The format was ready. The disciplined workflow — spec first, implementation second, validation automated — was always possible. What was missing was an implementer who would honor the spec without drifting.

William F. Nolan & George Clayton Johnson *Logan's Run · 1967*	***Units are assigned specific jobs. They perform those jobs for a set number of years and are then renewed.*** The three-year lifecycle principle draws its philosophical inspiration here. In a sealed utopian city, citizens are renewed at twenty-one — preventing the accumulation of legacy infrastructure that nobody wants to maintain but everyone is afraid to touch. The film adaptation (1976) extended this to thirty, apparently concluding that civilizations — unlike codebases — deserve a little more time before the inevitable renewal. The author wishes to note that in TCA, only the codebase is renewed. Developer retention policies remain humane.

Appendix: Repository Reference

The repository contains several reference documents that expand on concepts introduced in this paper. These are living documents, updated as TCA evolves. All are in the root of the repository.

Document	Purpose
`tca-guidelines.md` *TCA Project Guidelines*	The architectural rules and operational standards for any TCA implementation — contracts before code, Job isolation, three-year lifecycle, security posture, and delivery standards. Start here if you are implementing TCA on a new project.
`tca-project-kickoff.md` *Starting a TCA Project*	A thinking guide for the critical early phase before any code is written. Covers finding the Jobs, having the right conversations with your AI partner, and the mental shift from application-as-a-thing to application-as-a-set-of-things-that-happen.
`tca-contract-reference.md` *TCA Contract Reference*	Everything about contracts — OpenAPI 3.1 format, the two TCA extension blocks (x-tca-security and x-tca-observability), the rules that make contracts work as the lynchpin they are meant to be, and annotated examples from the proof-of-concept.
`tca-observability-reference.md` *Observability Job Reference*	Architecture, contract, implementation, and integration of the Observability Job. Covers the fire-and-forget reporting pattern, Redis pub/sub, path sanitization and filtering rules, and why observability graduated from development aid to required infrastructure.
`work-methods.md` *Engineer-AI Work Methods*	The operating reference for AI partners on how to work with the Engineer. Covers the partnership dynamic, delivery standards (tarball format, CHANGED_FILES.md, zero manual steps), the measure-twice rule, and the version-verification requirement before any implementation code is written. Designed as a template — personalize it for your own working style.
`tca-architecture.html` *Architecture Diagram*	Interactive visual reference showing all 12 Jobs, their languages, domain groupings, security boundaries, and data tier. Open in any browser — no server required. Use this as a companion when reading the contracts or walking through the codebase.

The proof-of-concept repository is at [github.com/Pelentan/tca-blackjack](). Every claim in this paper has a commit.